

算法设计与分析

Design & Analysis of Algorithms

第3章 动态规划

学习要点

- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性质 (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤
- 通过应用范例学习动态规划算法设计策略
 - (1) 基础题目
 - (2) 矩阵连乘问题
 - (3) 背包问题
 - (4) 子序列问题

3.1 动态规划算法概念-总体思想

- 动态规划算法与分治法类似，将待求解问题分解成若干个子问题。
- 分解得到的子问题往往不是互相独立的。子问题被重复计算了多次。
- 保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。

3.1 动态规划算法概念-基本步骤

- 问题结构分析

给出问题表示, 明确原始问题

- 递推关系建立

分析最优(子)结构, 构造递推公式

- 自底向上计算

确定计算顺序, 依次求解问题

- 最优方案追踪

记录决策过程, 输出最优方案

3.2 基础题目 exp 3-1. 斐波那契数(力扣509)

- **问题描述**：F(n) 表示，形成的序列称为斐波那契数列。该数列由 0 和 1 开始，后面的每一项数字都是前面两项数字的和。即： $F(0) = 0$, $F(1) = 1$ $F(n) = F(n - 1) + F(n - 2)$, 其中 $n > 1$ 给定n，请计算 F(n)。
- 示例：
 $F(2) = F(1) + F(0) = 1 + 0 = 1$
 $F(3) = F(2) + F(1) = 1 + 1 = 2$
 $F(4) = F(3) + F(2) = 2 + 1 = 3$

递归算法

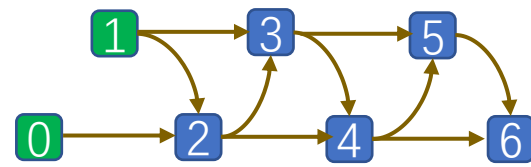
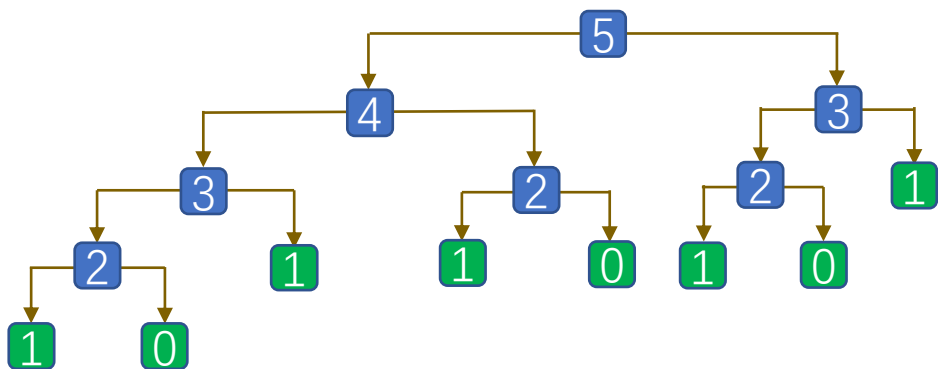
```
class Solution {  
public:  
    int fib(int N) {  
        if (N < 2) return N;  
        return fib(N - 1) + fib(N - 2);  
    }  
};
```

```
int fib(n) { return (2>n) ? n: fib(n-1)+fib(n-2)}
```

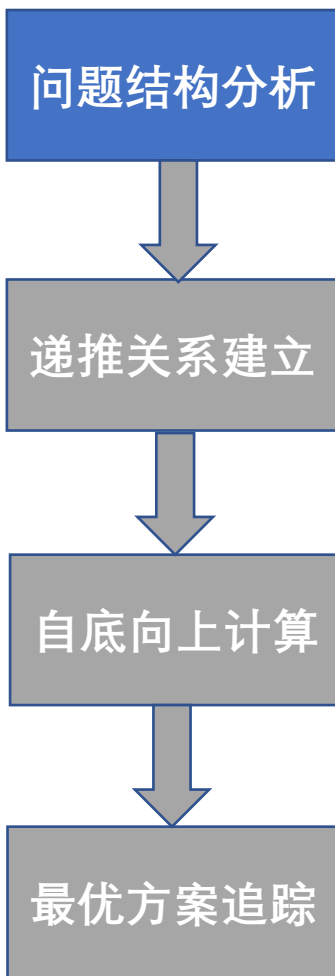
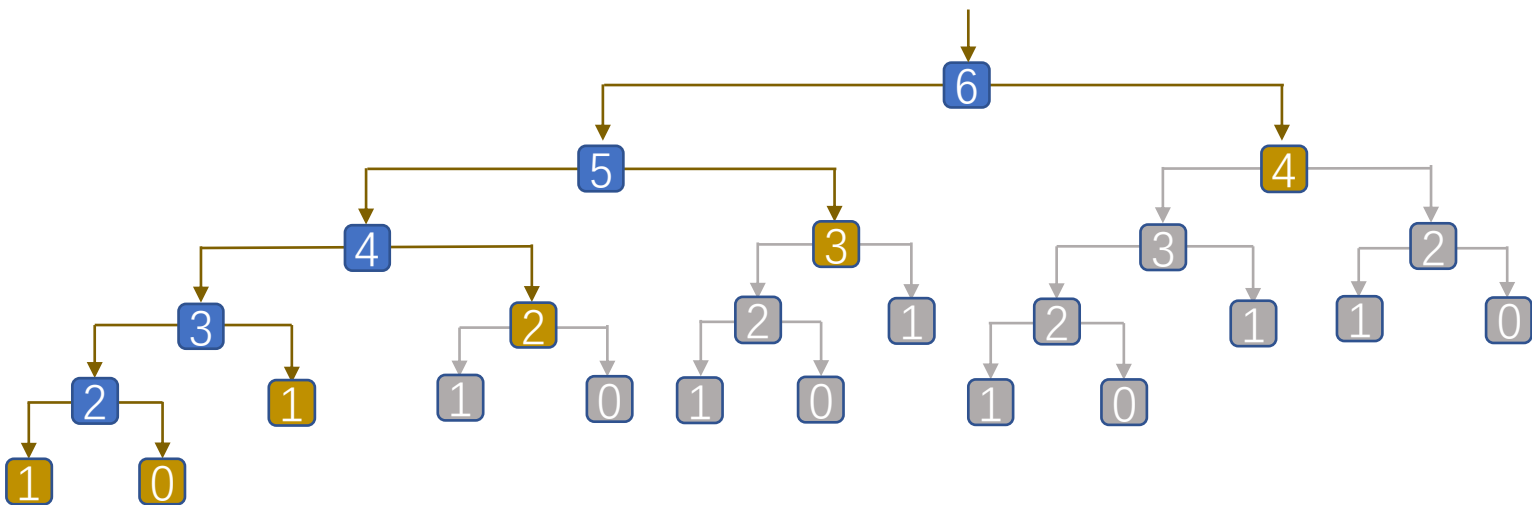
3.2 基础题目 exp 3-1. 斐波那契数(力扣509)

- 分析 $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

$$T(n) = T(n-1) + T(n-2) + 1 > 2 * T(n-2) + 1 = O(2^{n/2})$$



时间复杂度: $O(2^{n/2})$
空间复杂度: $O(n)$



3.2 基础题目 exp 3-1. 斐波那契数(力扣509)

- 确定dp数组以及下标的含义

dp[i]的定义为：第i个数的斐波那契数值是dp[i]

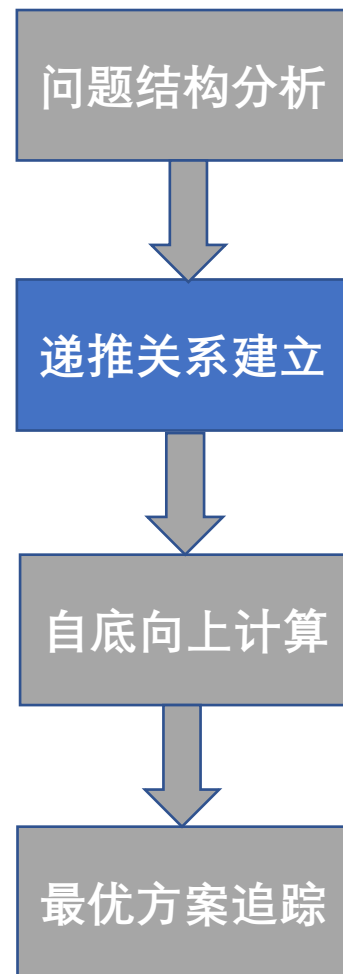
- 确定递推公式 状态转移方程

$dp[i] = dp[i - 1] + dp[i - 2];$

- dp数组如何初始化

$dp[0] = 0;$

$dp[1] = 1;$

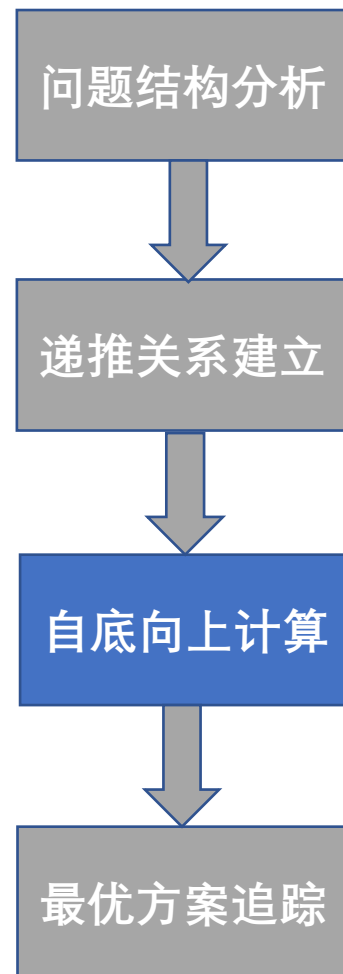


3.2 基础题目 exp 3-1. 斐波那契数(力扣509)

- 确定遍历顺序

从递归公式可以看出，遍历的顺序从前到后

```
for (int i = 2; i <= N; i++) {  
    dp[i] = dp[i - 1] + dp[i - 2];  
}
```



3.2 基础题目 exp 3-1. 斐波那契数(力扣509)

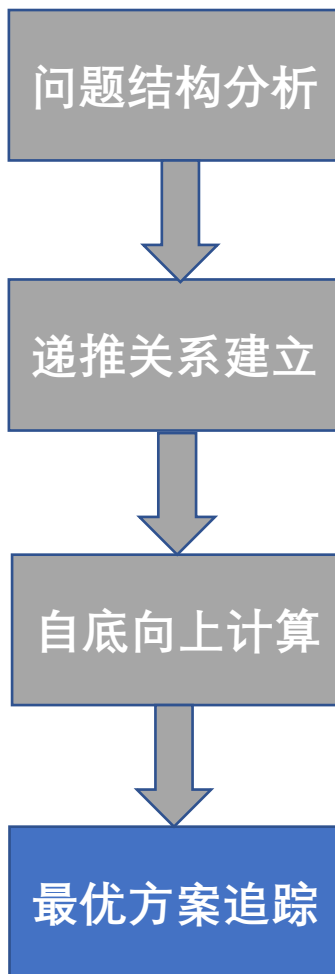
举例推导dp数组

按照这个递推公式，当N为10时dp数组：

0 1 1 2 3 5 8 13 21 34 55

```
class Solution {  
public:  
    int fib(int N) {  
        if (N <= 1) return N;  
        vector<int> dp(N + 1);  
        dp[0] = 0;  
        dp[1] = 1;  
        for (int i = 2; i <= N; i++) {  
            dp[i] = dp[i - 1] + dp[i - 2];  
        }  
        return dp[N];  
    }  
};
```

- 时间复杂度： $O(n)$
- 空间复杂度： $O(n)$



3.2 基础题目 exp 3-2. 爬楼梯(力扣70)

- **问题描述：**需要 n 阶才能到达楼顶。每次可以爬 1 或 2 个台阶。有多少种不同的方法可以爬到楼顶呢？

注意：给定 n 是一个正整数。

- 示例 1：

输入： 2

输出： 2

解释： 有两种方法可以爬到楼顶。

1 阶 + 1 阶

2 阶

- 示例 2：

输入： 3

输出： 3

解释： 有三种方法可以爬到楼顶。

1 阶 + 1 阶 + 1 阶

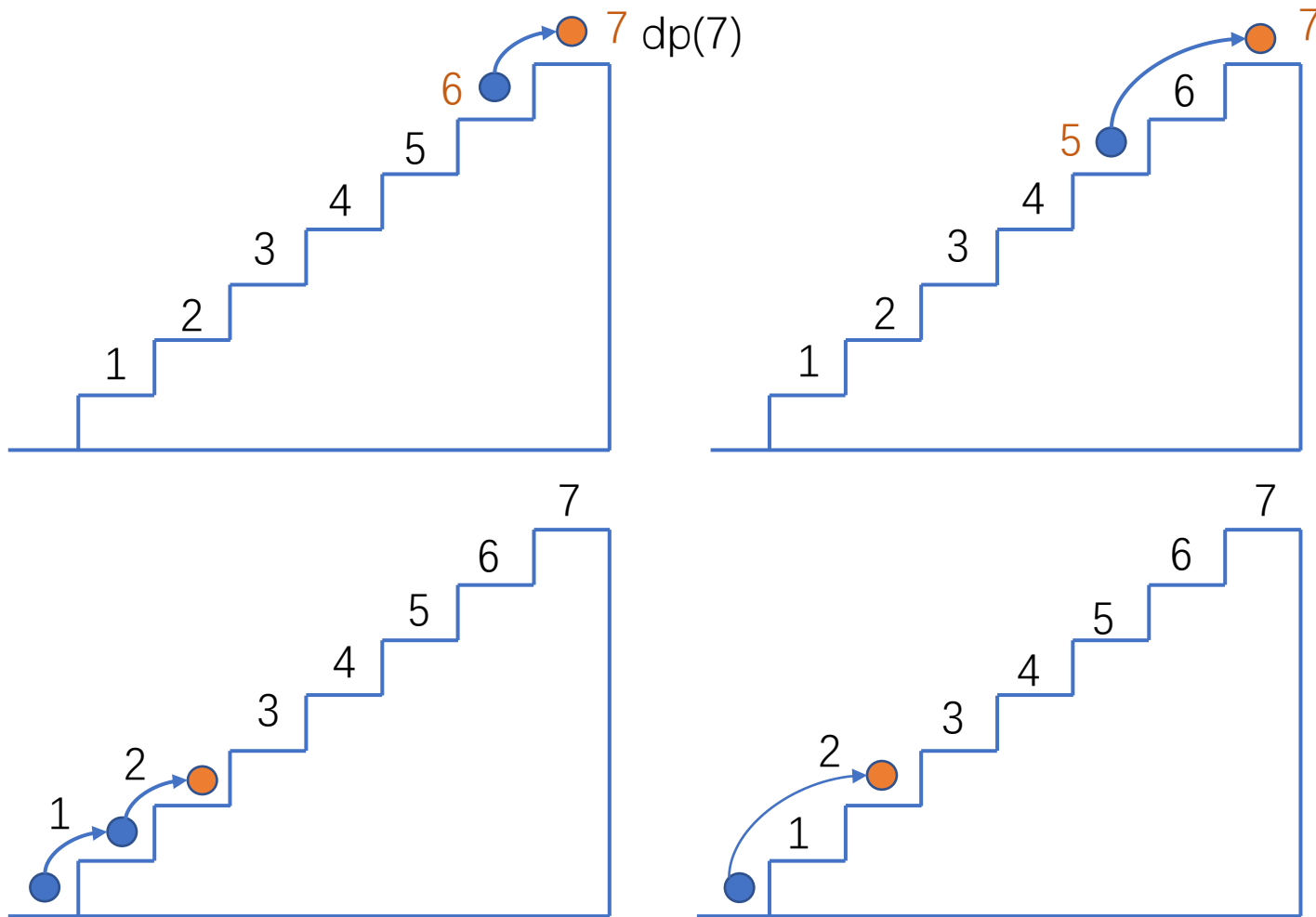
1 阶 + 2 阶

2 阶 + 1 阶

3.2 基础题目 exp 3-2. 爬楼梯(力扣70)

- 确定dp数组以及下标的含义

$$dp(7)=dp(6)+dp(5)$$



$$dp(2)=dp(1)+dp(0)=1+1$$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-2. 爬楼梯(力扣70)

- 确定dp数组以及下标的含义

dp[i]: 爬到第i层楼梯, 有dp[i]种方法。

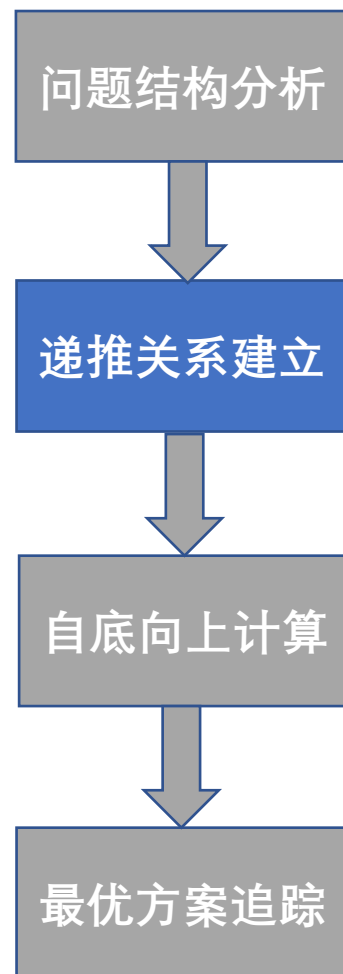
- 确定递推公式 状态转移方程

$dp[i] = dp[i - 1] + dp[i - 2];$

- dp数组如何初始化

$dp[0] = 1;$

$dp[1] = 1;$

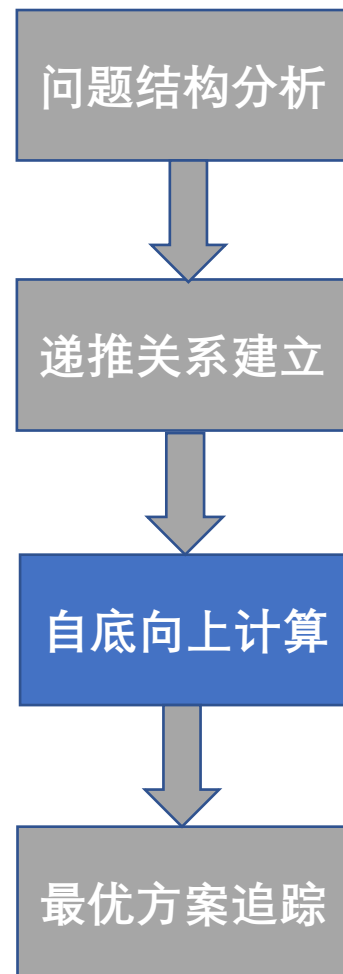


3.2 基础题目 exp 3-2. 爬楼梯(力扣70)

- 确定遍历顺序

遍历的顺序从小到大

```
for (int i = 2; i <= n; i++) { // 注意i是从2开始的  
    dp[i] = dp[i - 1] + dp[i - 2];  
}
```



3.2 基础题目 exp 3-2. 爬楼梯(力扣70)

- 举例推导dp数组

按照这个递推公式，当N为9的时dp数组：

1 1 2 3 5 8 13 21 34 55

// 版本一

```
class Solution {  
public:  
    int climbStairs(int n) {  
        if (n <= 0) return n; // 防止空指针  
        vector<int> dp(n + 1);  
        dp[0] = 1;  
        dp[1] = 1;  
        for (int i = 2; i <= n; i++) { // 注意i是从2开始的  
            dp[i] = dp[i - 1] + dp[i - 2];  
        }  
        return dp[n];  
    }  
};
```

•时间复杂度： $O(n)$

•空间复杂度： $O(n)$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

- **问题描述：**数组的每个下标作为一个阶梯，第 i 个阶梯对应着一个非负数的体力花费值 $\text{cost}[i]$ （下标从 0 开始）。每当你爬上一个阶梯你都要花费对应的体力值，一旦支付了相应的体力值，你就可以选择向上爬一个阶梯或者爬两个阶梯。请你找出达到楼层顶部的最低花费。在开始时，你可以选择从下标为 0 或 1 的元素作为初始阶梯。

- 示例 1:

输入: $\text{cost} = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]$

输出: 6 解释: 最低花费方式是从 $\text{cost}[0]$ 开始，逐个经过那些 1，跳过 $\text{cost}[3]$ ，一共花费 6。

- 提示:

cost 的长度范围是 $[2, 1000]$ 。

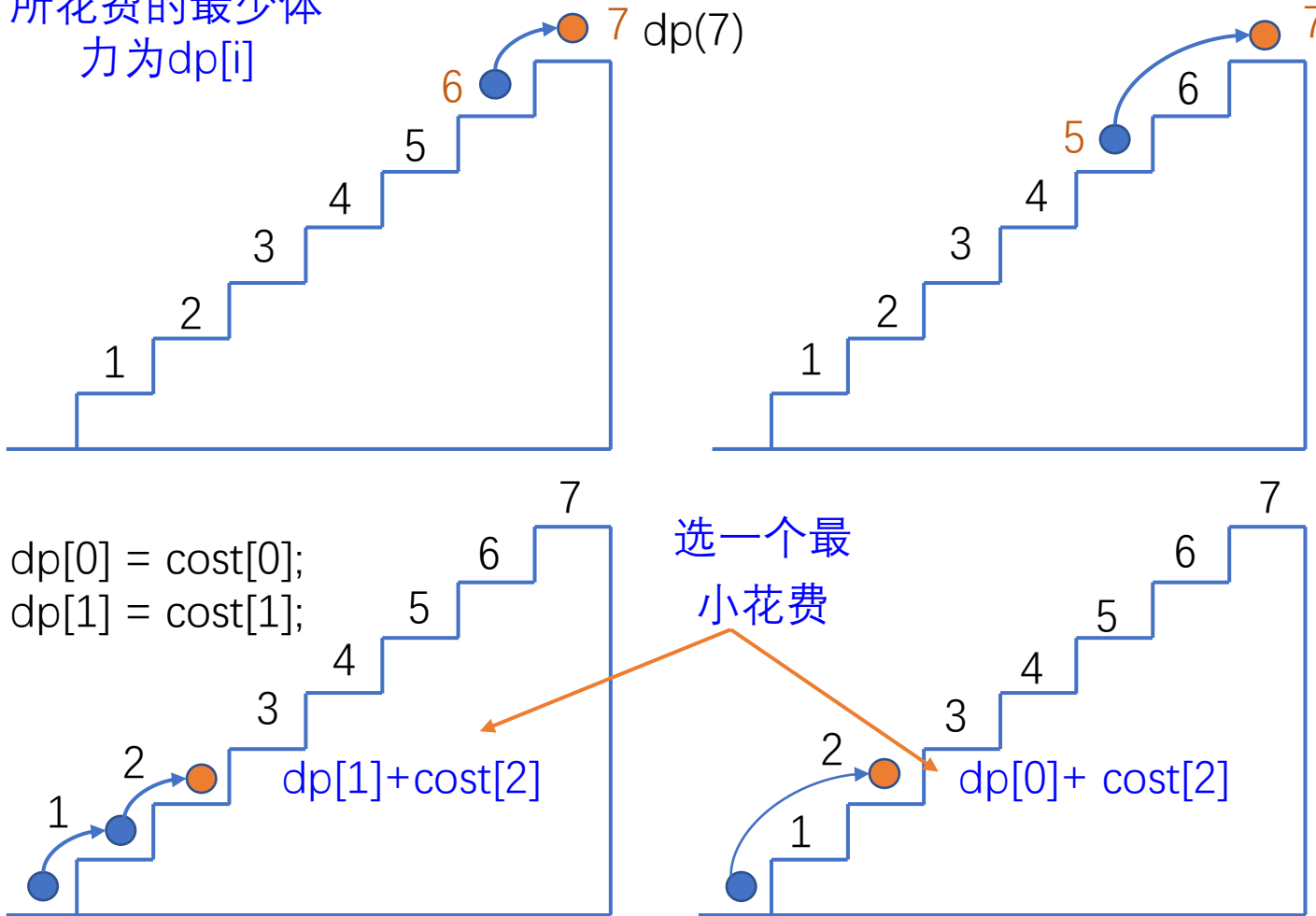
$\text{cost}[i]$ 将会是一个整型数据，范围为 $[0, 999]$ 。

3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

$\text{cost} = [1, 80, 1, 2, 1, 90, 1, 1, 100, 1]$

到达第*i*个台阶
所花费的最少体
力为 $\text{dp}[i]$

$$\text{dp}[7] = \min(\text{dp}[6], \text{dp}[5]) + \text{cost}[7]$$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

- 确定dp数组以及下标的含义

dp[i]的定义：到达第i个台阶所花费的最少体力为dp[i]。（注意这里认为是第一步一定是要花费）

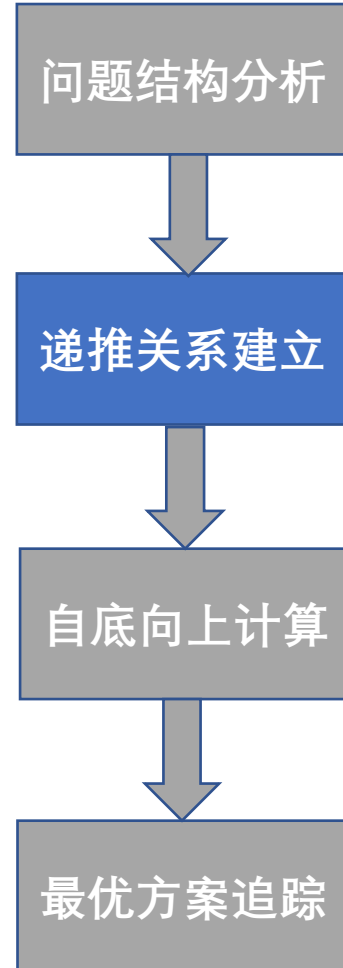
- 确定递推公式 状态转移方程

$$dp[i] = \min(dp[i - 1], dp[i - 2]) + cost[i];$$

- dp数组如何初始化

$$dp[0] = cost[0];$$

$$dp[1] = cost[1];$$

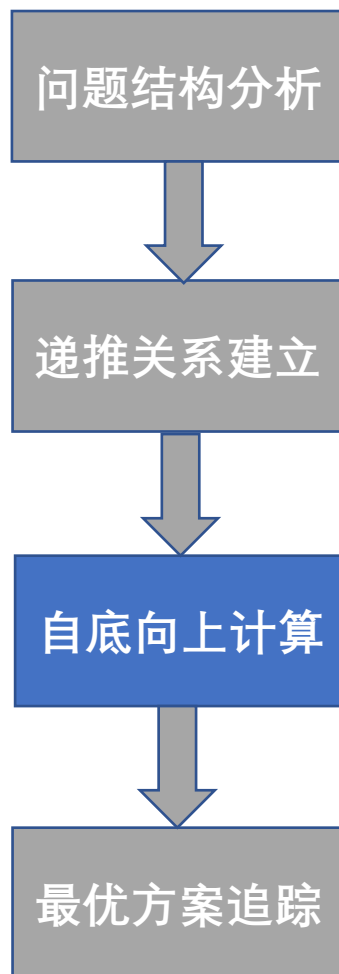


3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

- 确定遍历顺序

遍历的顺序从小到大

```
for (int i = 2; i < cost.size(); i++) {  
    dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];  
}
```



3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

- 举例推导dp数组

cost = [1, 80, 1, 2, 1, 90, 1, 1, 100, 1]

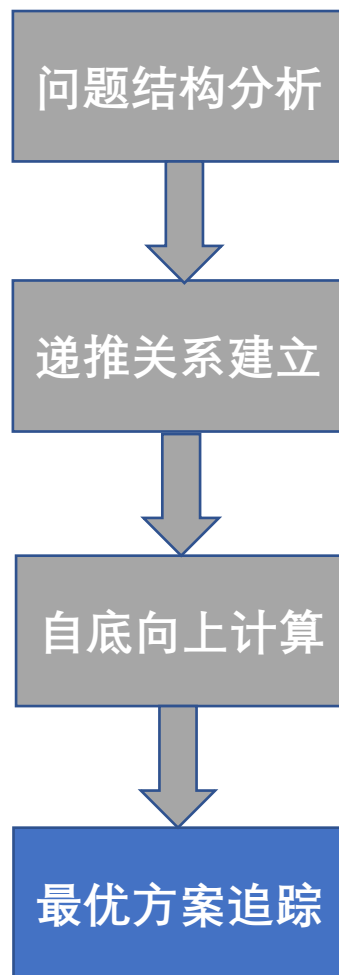
i	0	1	2	3	4	5	6	7	8	9
dp[i]	1	80	2	4	3	93	4	5	104	6

- 记录轨迹rec数组: 从哪个台阶过来的

i	0	1	2	3	4	5	6	7	8	9
rec[i]	0	0	0	2	2	4	4	6	6	7

反向求轨迹 9, 7, 6, 4, 2, 0

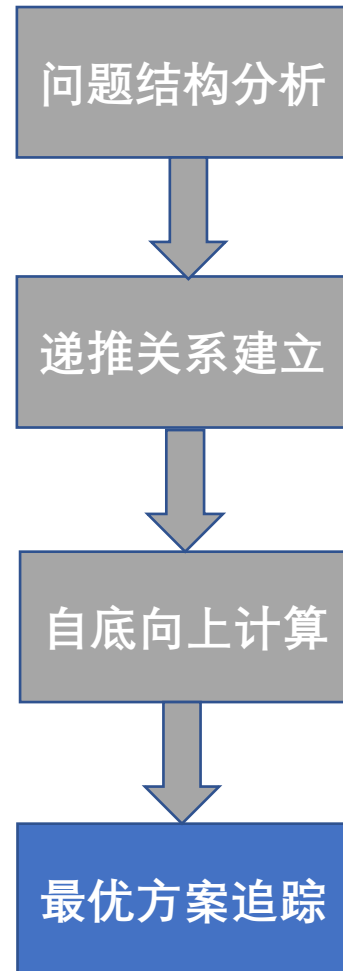
反向进栈, 正向出栈



3.2 基础题目 exp 3-3. 使用最小花费爬楼梯(力扣746)

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        vector<int> dp(cost.size());
        dp[0] = cost[0];
        dp[1] = cost[1];
        for (int i = 2; i < cost.size(); i++) {
            dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];
        }
        // 注意最后一步可以理解为不用花费，所以取倒数
        // 第一步，第二步的最少值
        return min(dp[cost.size() - 1], dp[cost.size() - 2]);
    }
};
```

- 时间复杂度: $O(n)$
- 空间复杂度: $O(n)$



实验3-1：使用最小花费爬楼梯

- 要求
 1. 给出最小花费
 2. 给出所走的台阶编号

3.2 基础题目 exp 3-4. 不同路径(力扣62)

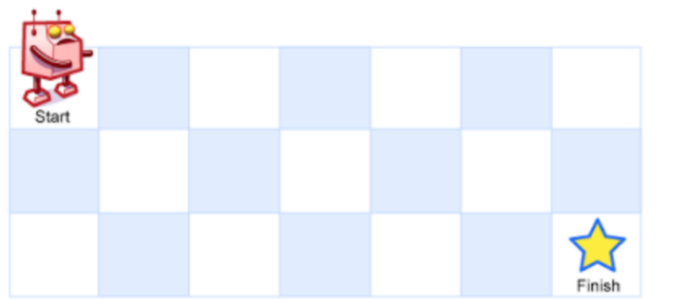
- **问题描述：** 一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为 “Start”）。机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish”）。问总共有多少条不同的路径？

- 示例 1：

输入： $m = 3, n = 7$

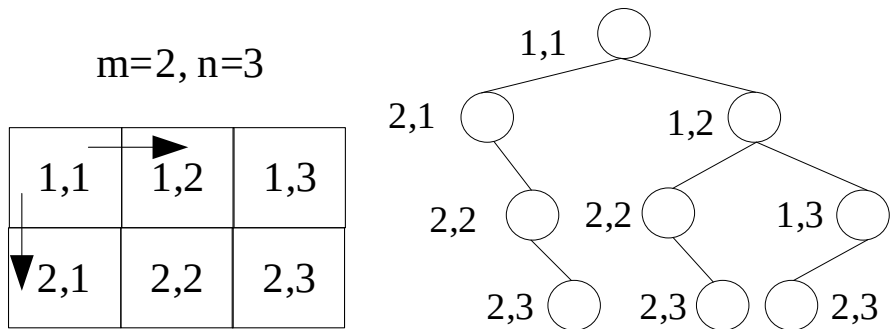
输出： 28

示例 1：



3.2 基础题目 exp 3-4. 不同路径(力扣62)

- **递归实现：** 机器人走过的路径可以抽象为一棵二叉树，而叶子节点就是终点。可以转化为求二叉树叶子节点的个数，采用递归遍历二叉树。



- **时间复杂度：**遍历整个二叉树，树的深度 $m+n-1$ （深度按从1开始计算）。

递归调用次数为 $2^{(m+n-1)} - 1$ 。

时间复杂度为 $O(2^{(m+n-1)} - 1)$ 。

```
class Solution {
private:
    int dfs(int i, int j, int m, int n) {
        if (i > m || j > n) return 0; // 越界了
        if (i == m && j == n) return 1; // 找到
        一种方法， 相当于找到了叶子节点
        return dfs(i + 1, j, m, n) + dfs(i, j + 1,
m, n);
    }
public:
    int uniquePaths(int m, int n) {
        return dfs(1, 1, m, n);
    }
};
```

3.2 基础题目 exp 3-4. 不同路径(力扣62)

从(0, 0) 位置, 到(m - 1, n - 1)终点

- 确定dp数组以及下标的含义

$dp[i][j]$: 从 (0, 0) 出发, 到(i, j) 有 $dp[i][j]$ 条不同的路径。

m=3, n=7

1	1	1	1	1	1	1
1	2	3	4	5	6	7
1	3	6	10	15	21	28

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-4. 不同路径(力扣62)

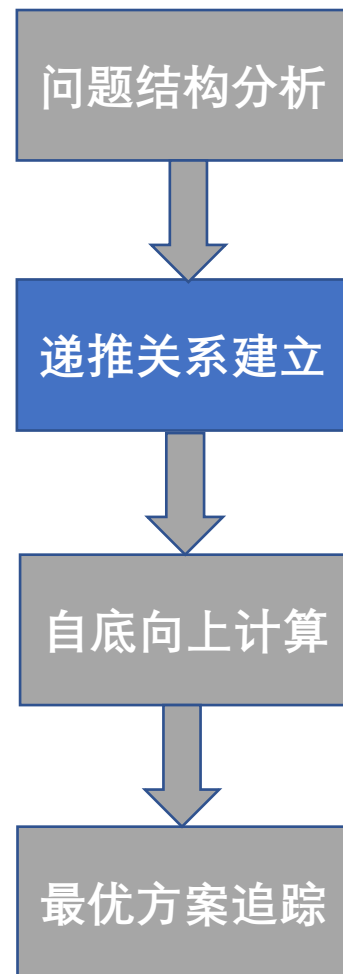
从(0, 0) 位置, 到(m - 1, n - 1)终点

- 确定dp数组以及下标的含义

$dp[i][j]$: 从 (0, 0) 出发, 到(i, j) 有 $dp[i][j]$ 条不同的路径。

- 确定递推公式 状态转移方程

$dp[i][j] = dp[i - 1][j] + dp[i][j - 1];$



3.2 基础题目 exp 3-4. 不同路径(力扣62)

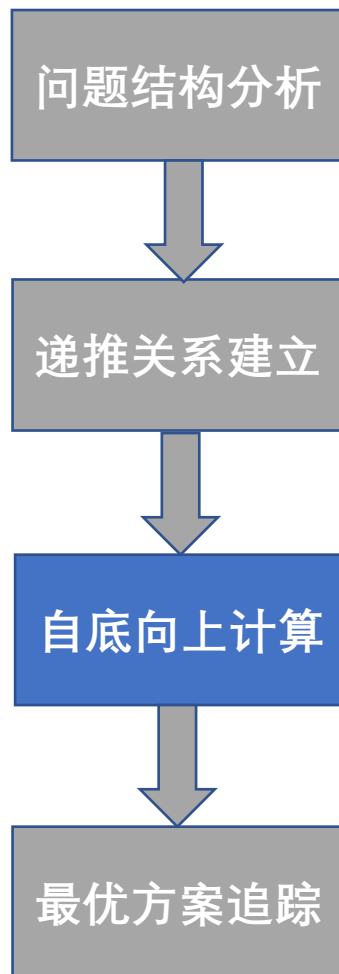
- dp数组如何初始化

```
for (int i = 0; i < m; i++) dp[i][0] = 1;
```

```
for (int j = 0; j < n; j++) dp[0][j] = 1;
```

- 确定遍历顺序

从左到右一层一层遍历, 推导 $dp[i][j]$ 时, $dp[i - 1][j]$ 和 $dp[i][j - 1]$ 一定是有数值



3.2 基础题目 exp 3-4. 不同路径(力扣62)

m=3, n=7

1	1	1	1	1	1	1
1	2	3	4	5	6	7
1	3	6	10	15	21	28

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, 0));
        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
        return dp[m - 1][n - 1];
    }
};
```

- 时间复杂度: $O(m \times n)$
- 空间复杂度: $O(m \times n)$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-4. 不同路径(力扣62)

- 一维数组实现，滚动数组。替换上一行。

m=3, n=7

1	1	1	1	1	1	1
1	2	3	4	5	6	7
1	3	6	10	15	21	28

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n);
        for (int i = 0; i < n; i++) dp[i] = 1;
        for (int j = 1; j < m; j++) {
            for (int i = 1; i < n; i++) {
                dp[i] += dp[i - 1];
            }
        }
        return dp[n - 1];
    }
};
```

- 时间复杂度: $O(m \times n)$
- 空间复杂度: $O(n)$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.2 基础题目 exp 3-5. 不同路径II (力扣63)

- 一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为“Start”）。机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。问总共有多少条不同的路径？现在考虑网格中有障碍物。

- 示例 1:

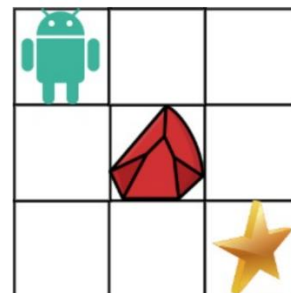
输入: `obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]`

输出: 2 解释:

3x3 网格的正中间有一个障碍物。

从左上角到右下角一共有 2 条不同的路径:

1. 向右 -> 向右 -> 向下 -> 向下
2. 向下 -> 向下 -> 向右 -> 向右



3.2 基础题目 exp 3-5. 不同路径II (力扣63)

从(0, 0) 位置, 到(m - 1, n - 1)终点

1. 确定dp数组以及下标的含义

$dp[i][j]$: 从 (0, 0) 出发, 到(i, j) 有 $dp[i][j]$ 条不同的路径。

2. 确定递推公式 状态转移方程

如果是障碍应该保持初始状态0

```
if (obstacleGrid[i][j] == 0) {
```

```
//当(i, j)没有障碍, 再推导dp[i][j]
```

```
    dp[i][j] = dp[i - 1][j] + dp[i][j - 1];}
```

3. dp数组如何初始化

```
vector<vector<int>> dp(m, vector<int>(n, 0));
```

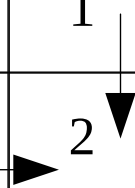
```
for (int i=0; i < m && obstacleGrid[i][0]==0; i++) dp[i][0]=1;
```

```
for (int j=0; j < n && obstacleGrid[0][j]==0; j++) dp[0][j] =1;
```

4. 确定遍历顺序

```
for (int i = 1; i < m; i++) {  
    for (int j = 1; j < n; j++) {  
        if (obstacleGrid[i][j] == 1) continue;  
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1]; } }
```

1	1	1
1	0	1
1	1	2



实验3-2：走格子求最小（大）路径和

- 有一个由数字组成的规格为 $n \times m$ 的矩阵，初始在左上角，要求每次只能向下或向右移动，问该数字矩阵从最左上角到最右下角的最小路径和是多少？（路径和就是将某路径中的所有权值全部加起来的总和）

- 示例 1：

输入：

1	3	5	7
---	---	---	---

输出：

8	6	4	2
---	---	---	---

5	0	1	3
---	---	---	---

4	8	7	2
---	---	---	---